

Chapter 2

Distributed Objects and Remote Communication

Prepared By: Er. Amrit Poudel

Interprocess Communication

- It is the communication between two or more different processes in the distributed system
- In order for one process to communicate with another, one process sends a message(a sequence of bytes) to a destination and another process at the destination receives a message.
- This activity involves the communication of data from the sending process to the receiving process and may involve the synchronization of the two processes

Inter-Process Communication (IPC)



Characteristics of Distributed System

- Synchronous and Asynchronous Communication
- Message Destinations
- Reliability
- Ordering

Synchronous and Asynchronous Communication

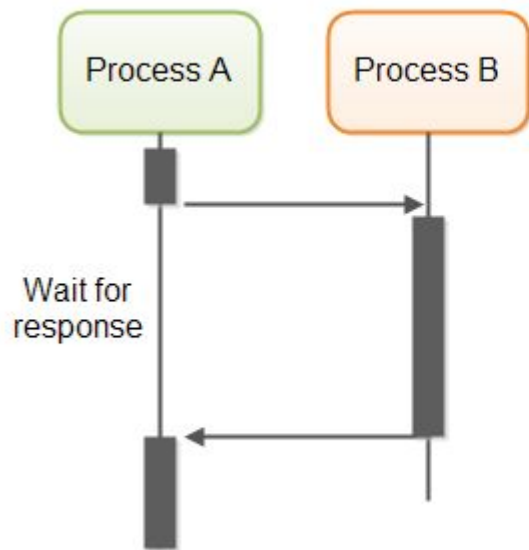
Synchronous

- Sender blocks until message received.
 - Often sender blocked until message is received and a reply received.
- Sender and receiver must be active at the same time
- Client-Server generally uses synchronous communication

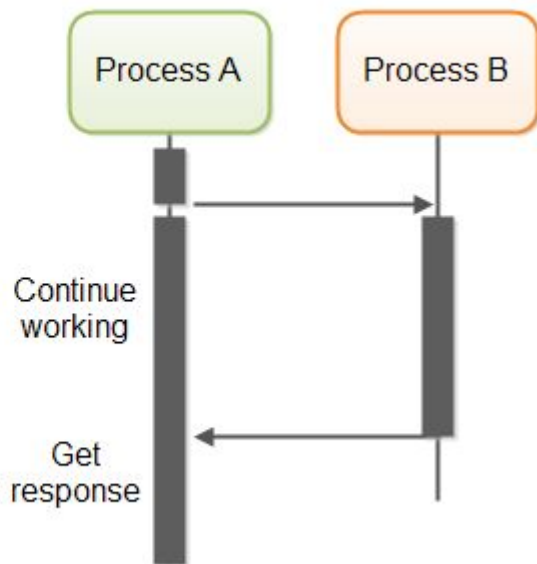
Asynchronous:

- Sender continues execution after sending message(does not block waiting for reply)
- Message may be queued if receiver not active
- Message may be processed later at receiver's convenience

Synchronous



Asynchronous



Message Destinations

- Message are sent to (*Internet address, local port*) pairs
- An IP address is a unique address that identifies a device on the internet or a local network.
- A local port is a message destination within a computer, specified as an integer.
- A port has an exactly one receiver but can have many senders
- An alternative to ports is that messages should be addressed to processes.

Reliability

- Reliable communication is defined in terms of **validity** and **integrity**
- **Reliability**
 - A point-to-point message service can be described as reliable if messages are guaranteed to be delivered despite a 'reasonable' number of packets being dropped or lost.
- **Integrity**
 - For integrity, messages must arrive uncorrupted and without duplication.
- **Validity** deals with accuracy of message

Ordering

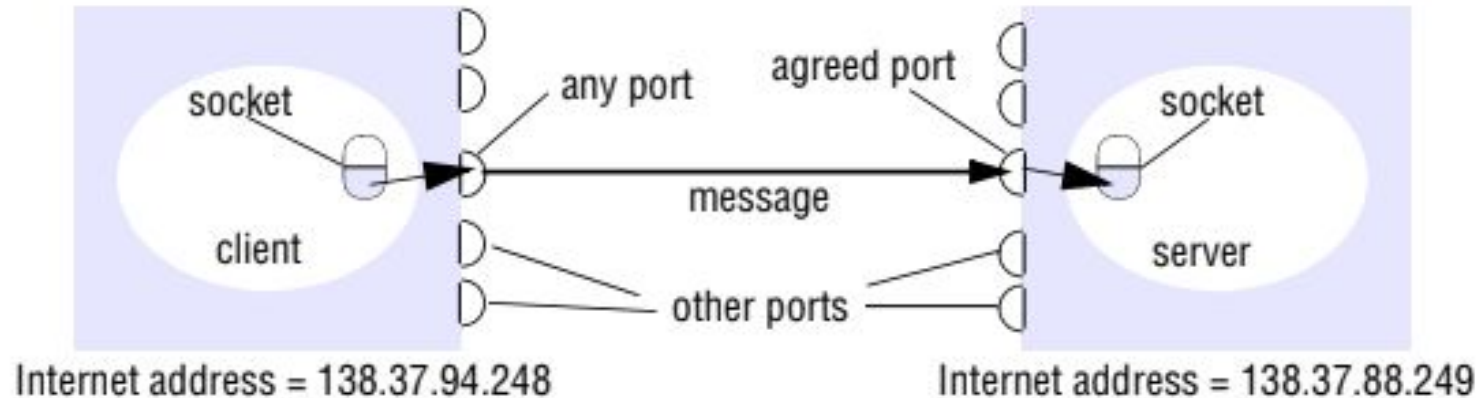
- Some applications require that messages be delivered in sender order-that is, the order in which they were transmitted by the sender.
- The delivery of messages out of sender order is regarded as a failure by such applications.

Sockets

- Both forms of communication(UDP and TCP) use the *socket* abstraction,
- Provides an endpoint for communication between processes.
- Interprocess communication consists of transmitting a message between a socket in one process and a socket in another process.
- Processes may use same socket for sending and receiving messages.

Sockets

- For a process to receive messages, its socket must be bound to local port and one of the internet addresses of the computer on which it runs.
- Message sent to a particular Internet address and port number can be received only by a process whose socket is associated with that Internet address and port number.



UDP datagram communication

- UDP datagram communication UDP (User Datagram Protocol) is connectionless (UDP) protocol.
- A datagram sent by UDP is transmitted from a sending process to a receiving process without acknowledgement.
- If a failure occurs, the message may not arrive.
- A datagram is transmitted between processes when one process sends it and another receives it.

UDP datagram communication

- To send or receive messages a process must first create a socket bound to an Internet address of the local host and a local port.
- A server will bind its socket to a server port – one that it makes known to clients so that they can send messages to it.
- A client binds its socket to any free local port.
- The receive method returns the Internet address and port

Some issues related to datagram communication

- **Message size:** the receiving process needs to specify an array of bytes of a particular size. (message size is restricted)
- **Blocking:** non-blocking sends and blocking receives
- **Timeouts :** timeouts can be set on sockets and it should be fairly large in comparison with the time required to transmit a message.
- **Receive from any :** invocation of receive method gets a message addressed to its socket from any origin.

Failure Model

- **Ordering :** out of order
- **Omission failures :** messages could be dropped
 - Send-omission failure
 - Receive Omission Failure

TCP Stream Communication

- TCP (Transmission Control Protocol) is a connection-oriented protocol
- The API to the TCP protocol provides the abstraction of a stream of bytes to which data may be written and from which data may be read.

The following characteristics of the network are hidden by the stream abstraction:

Message size :

- **Message size :** The application can choose how much data it writes to a stream or reads from it.

TCP Stream Communication

- **Lost message :** using ACK scheme
- **Flow control :** The TCP protocol attempts to match the speeds of the processes that read from and write to a stream.
- **Message duplication and ordering :** Message identifiers are associated with each IP packet, which enables the recipient to detect and reject duplicates, or to reorder messages that do not arrive in sender order
- **Message destination :**
 - A pair of communicating processes establish a connection before they can communicate over a stream.
 - Once a connection is established, the processes simply read from and write to the stream without needing to use.

The API for stream communication using TCP

- The API for stream communication assumes that when a pair of processes are establishing a connection, one of them plays the client role and the other plays the server role.
- **The client role** involves creating a stream socket bound to any port and then making a connect request asking for a connection to a server at its server port.
- **The server role** involves creating a listening socket bound to a server port and waiting for clients to request connections
- The listening socket maintains a queue of incoming connection requests.

Some issues related to stream communication

Matching of data items :

- two communicating processes need to agree as to contents of the data transmitted over a stream.

Blocking :

- The data written to a stream is kept in a queue at the destination socket.
- When a process attempts to read data from an input channel, it will get data from the queue or it will block until data becomes available.
- The process that writes data to a stream may be blocked by the TCP flow-control mechanism if the socket at the other end is queuing as much data as the protocol allows

Issues related to stream communication:

Threads:

- When a server accepts a connection, it generally creates a new thread in which to communicate with the new client.
- The advantage of using a separate thread for each client is that the server can block when waiting for input without delaying other clients.
- In an environment in which threads are not provided, an alternative is to test whether input is available from a stream before attempting to read it;

TCP stream communication

Failure model

- To satisfy the **integrity** property of reliable communication, TCP streams use checksums to detect and reject corrupt packets and sequence numbers to detect and reject duplicate packets.
- For the sake of the **validity** property, TCP streams use timeouts and retransmissions to deal with lost packets. Therefore, messages are guaranteed to be delivered even when some of the underlying packets are lost.

TCP stream communication

Use of TCP

- HTTP (The Hypertext Transfer Protocol)
- FTP
- Telnet
- SMTP

External data representation and marshalling

External data representation:

- An agreed standard for the representation of data structures and primitives values

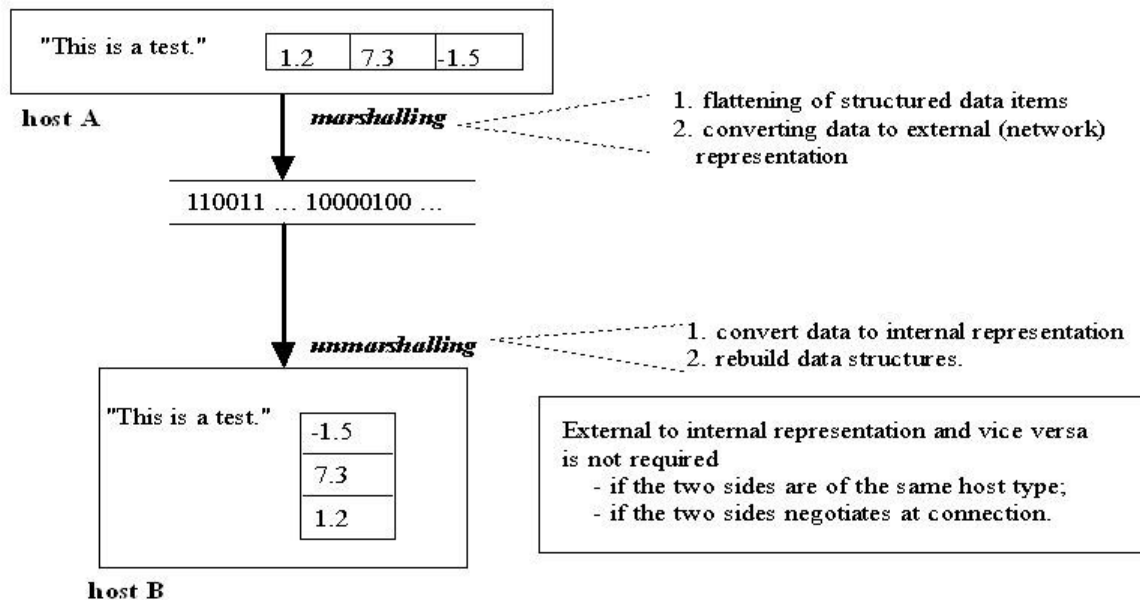
Marshalling

- Marshalling Is the process of taking a collection of data items and assembling them into a form suitable for transmission in a message

Unmarshalling

- Is the process of disassembling data items on arrival to produce an equivalent collection of data items at the destination

Data Marshalling



Approaches to external data representation and marshalling



- **CORBA's common data representation,**
 - CORBA is a specification developed by Object Management Group (OMG)
 - Its a specification for creating, distributing, and managing objects in distributed networks.
 - describes a messaging mechanism by which objects distributed over a network can transfer messages with each other irrespective of the platform or language used to create those objects.
- **Java's object serialization,**
 - In Java remote method invocation (RMI), both objects and primitive data values may be passed as arguments and results of method invocations.
 - In Java, the term serialization refers to the activity of flattening an object(An instance of a class) or a connected set of objects into a serial form that is suitable for storing on disk or transmitting in a message.

Approaches to external data representation and marshalling

XML

- XML is a markup language that was defined by the World Wide Web Consortium for general use on the web.
- XML was initially developed for writing structured documents for the web.
- XML is used to enable clients to communicate with web services and for defining the interfaces and other properties of web services.

CORBA CDR message

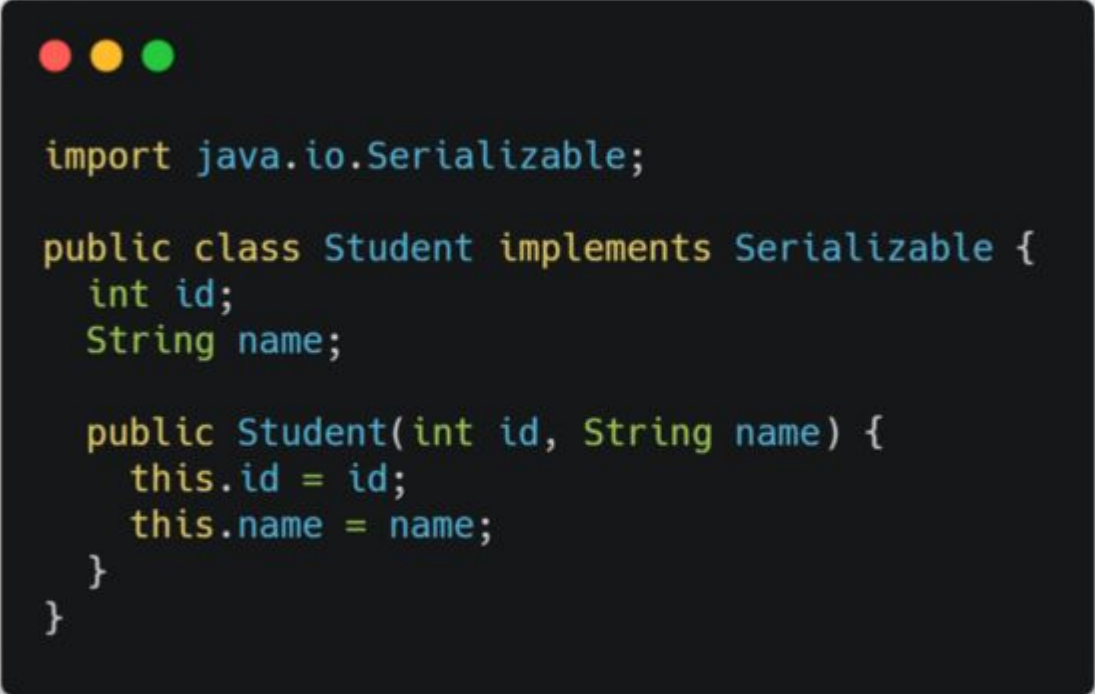
```
Struct Person {  
    string name;  
    string place;  
    long year;  
};
```

<i>index in sequence of bytes</i>	<i>4 bytes</i>	<i>notes on representation</i>
0-3	5	<i>length of string</i>
4-7	"Smit"	<i>'Smith'</i>
8-11	"h__"	
12-15	6	<i>length of string</i>
16-19	"Lond"	<i>'London'</i>
20-23	"on__"	
24-27	1934	<i>unsigned long</i>

The flattened form represents a *Person* struct with value: {'Smith', 'London', 1934}



Java Object Serialization Example



```
import java.io.Serializable;

public class Student implements Serializable {
    int id;
    String name;

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

Xml Data Representation Example



```
<person>  
  <id>1</id>  
  <name>Guy Fawkes</name>  
</person>
```

Request Reply Communication

- Designed to support the roles and message exchanges in client server interactions
- Synchronous because the process blocks until the reply arrives from the server.

Request Reply Protocol

- Matches request to replies
- Based on trio of communication primitives, *doOperation*, *getRequest* and *sendReply*
- The ***doOperation*** method is used by clients to invoke remote operations
 - Its arguments specify the remote server and which operation to invoke, together with additional information required by the operation
- The ***getRequest*** is used by a server process to acquire service requests.
- The ***sendReply*** is used to send the reply message to the client.
- When the reply message is received by the client the original ***doOperation*** is unblocked and execution of the client program continues

Figure 5.2
Request-reply communication

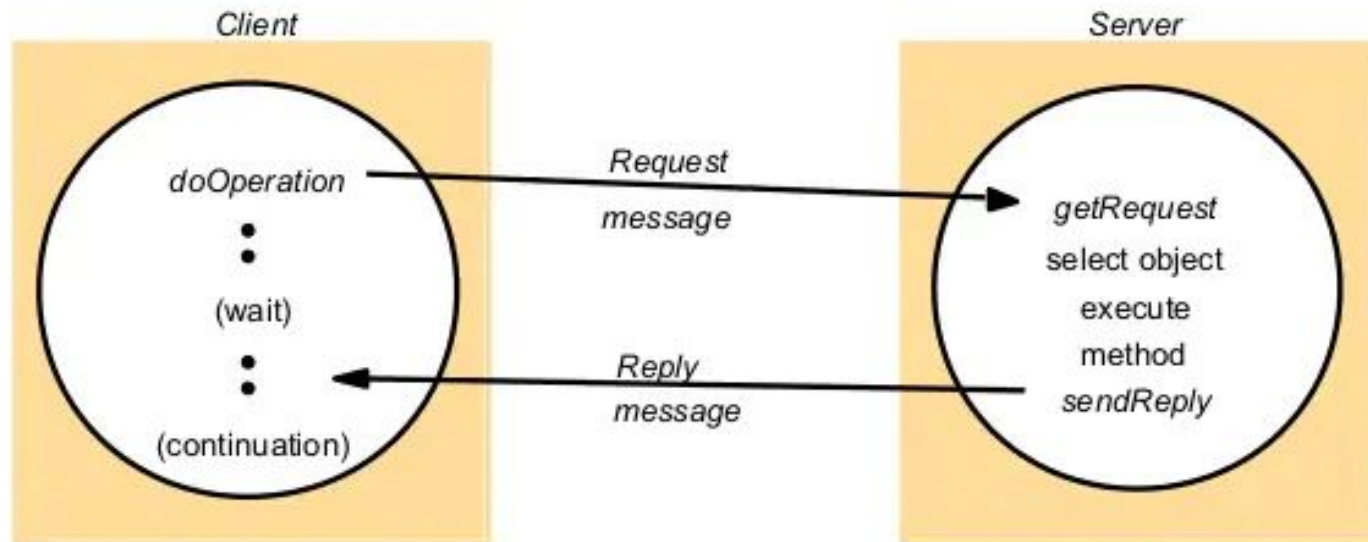


Figure 5.4
Request-reply message structure

messageType	<i>int (0=Request, 1= Reply)</i>
requestId	<i>int</i>
remoteReference	<i>RemoteRef</i>
operationId	<i>int or Operation</i>
arguments	<i>array of bytes</i>

Programming Models in Distributed System

- **RPC (Remote Procedure Call):**

- the earliest and perhaps the best-known programming model for distributed programming
- It *allows client programs to call procedures in server programs* running in separate processes and generally in different computers from the client

- **RMI (Remote Method Invocation)**

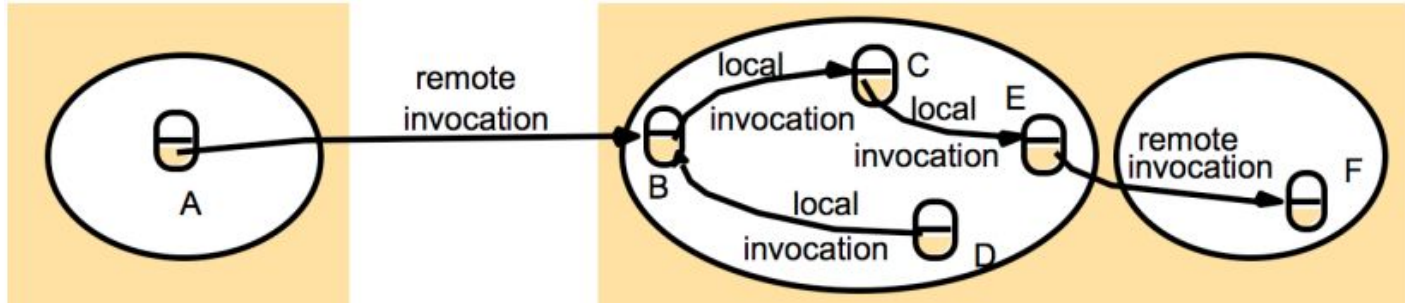
- extension of local method invocation of object-oriented programming.
- It *allows an object living in one process to invoke the methods of an object living in another process*
- Most famous example: Java RMI

- **Event-Based Programming:**

- allows objects to receive notification of the events at other objects in which they have registered interest.

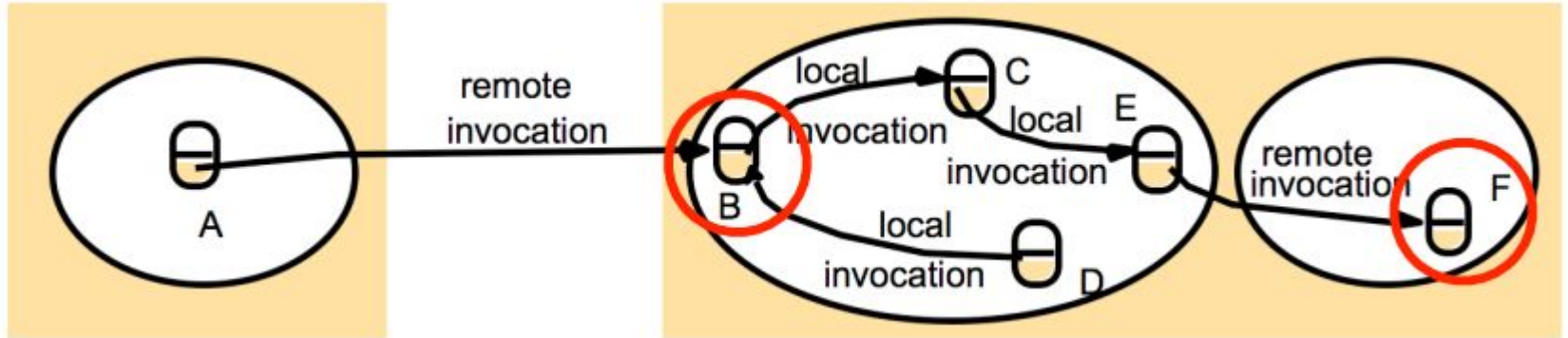
The Distributed Object Model

- Each process contains a **collection of objects**
 - some of which can receive both **local** and **remote** invocations
 - whereas the other objects can receive **only local** invocations
- **Method invocations between objects in different processes, whether in the same computer or not**, are known as **remote method invocations**.
- **Method invocations between objects in the same process** are **local method invocations**.



Remote Objects

- **Remote objects:** objects that can receive **remote invocations**.
- **Fundamental concepts** of the distributed object model:
 - Objects can invoke the methods of a remote object if they have access to its **remote object reference**.
 - Every remote object has a **remote interface** that specifies which of its methods can be invoked remotely



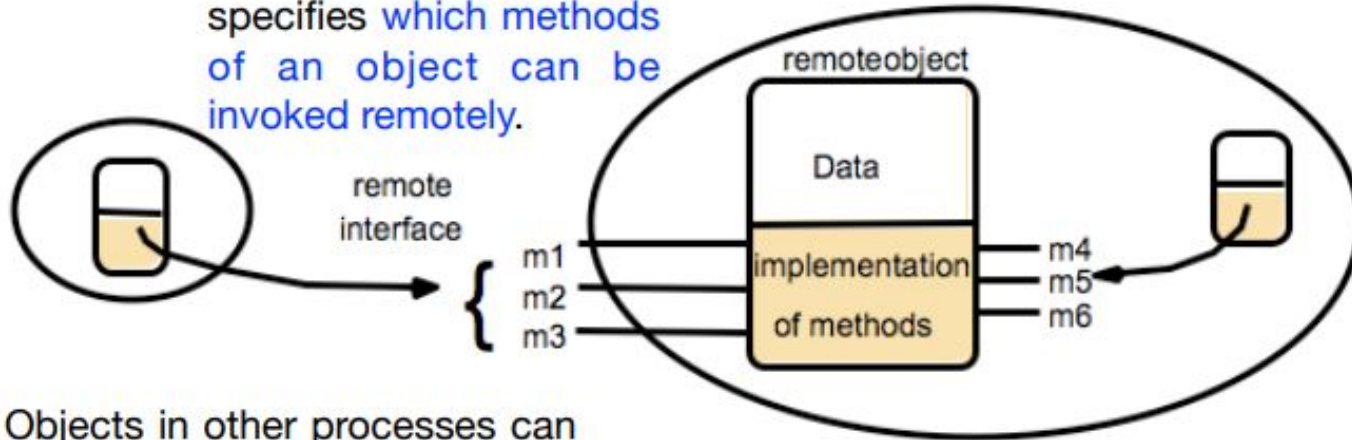
Remote Object Reference

- A **remote object reference** is an *identifier* that can be used throughout a distributed system to refer to a *particular unique remote object*.
- A remote object reference is passed in the invocation message to specify **which object is to be invoked**
- Remote object references are analogous to local ones in that:
 - the remote object to receive a remote method invocation is specified by the invoker as a remote object reference
 - remote object references may be passed as arguments and results of remote method invocations.

Remote Interface

The **remote interface** specifies **which methods of an object can be invoked remotely**.

The class of a remote object implements the methods of its remote interface.

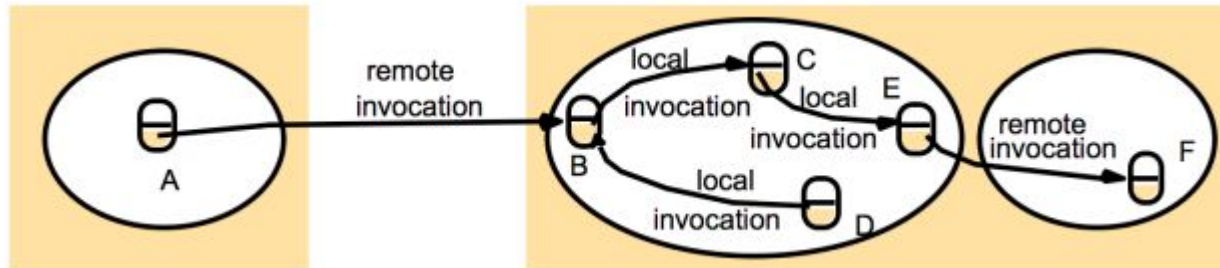


Objects in other processes can invoke *only* the methods that belong to the remote interface of a remote object.

Local objects can invoke the methods in the remote interface as well as other methods implemented by a remote object.

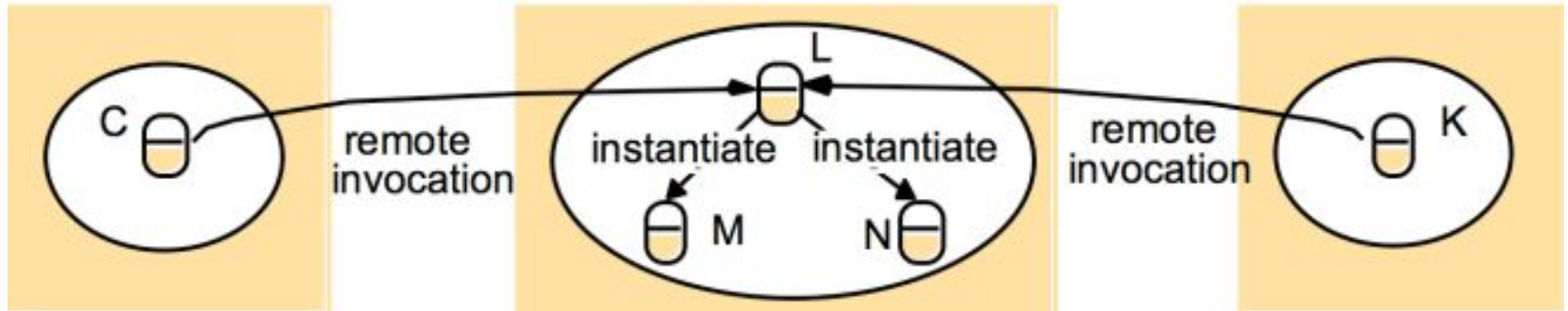
Actions in a Distributed Object System

- An **action** is initiated by a method invocation, which may result in further invocations on methods in other objects.
- The objects involved in a chain of related invocations may be **located in different processes or different computers**
- When an invocation crosses the boundary of a process or computer, RMI is used and the remote reference of the object must be available to the invoker
- Remote object references may be obtained as the results of remote method invocations (example: A might obtain a remote reference to F from B)



Creation of Remote Objects

- When an action leads to the instantiation of a new object, that object will normally live within the process where the instantiation is requested.
- If a newly instantiated object has a remote interface, it will be a remote object with a remote object reference.



Exceptions

- Any remote invocation may fail for reasons related to the invoked object being in a different process or computer from the invoker.

Example: the process containing the remote object may have crashed or may be too busy to reply, or the invocation or result message may be lost.

- Remote method invocation should be able to raise exceptions!
 - Timeouts that are due to distribution
 - Exceptions raised during the execution of the method invoked:
 - attempt to read beyond the end of a file
 - attempt to access a file without the correct permissions
 -

RMI Invocation Semantics

- In Request-reply protocols, *doOperation* can be implemented in different ways to provide different **delivery guarantees**
- These choices lead to a variety of possible **semantics** for the reliability of remote invocations as seen by the invoker.

Main Choices for Implementing RMI

- **Retry request message:** whether to retransmit the request message until either a reply is received or the server is assumed to have failed
- **Duplicate filtering:** when retransmissions are used, whether to filter out duplicate requests at the server.
- **Retransmission of results:** whether to keep a history of result messages to enable lost results to be retransmitted without re-executing the operations at the server
- Combination of these choices lead to a variety of possible **semantics** for the reliability of remote invocations: **Maybe, At-least-once, At-most-once**

RMI Invocation Semantics

<i>Fault tolerance measures</i>			<i>Invocation semantics</i>
<i>Retransmit request message</i>	<i>Duplicate filtering</i>	<i>Re-execute procedure or retransmit reply</i>	
No	Not applicable	Not applicable	<i>Maybe</i>
Yes	No	Re-execute procedure	<i>At-least-once</i>
Yes	Yes	Retransmit reply	<i>At-most-once</i>

Maybe: Implies uncertainty in reliability; the system can't guarantee whether the operation was successful or not.

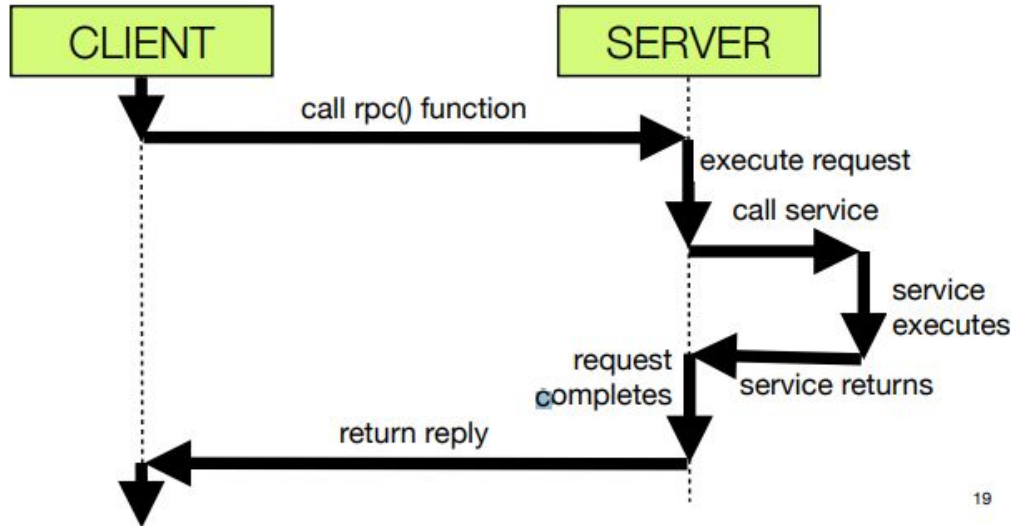
At-least-once: Ensures that the operation will be performed at least once, potentially leading to duplicate results but guaranteeing that the operation is executed.

At-most-once: Guarantees that the operation is performed at most once, ensuring no duplicates at the expense of potential omission if there's a failure.

- In **Java RMI** the invocation semantics is *at-most-once*.
- In **CORBA** is *at-most-once* but *maybe* semantics can be requested for methods that do not return results.

Remote Procedure Call (RPC)

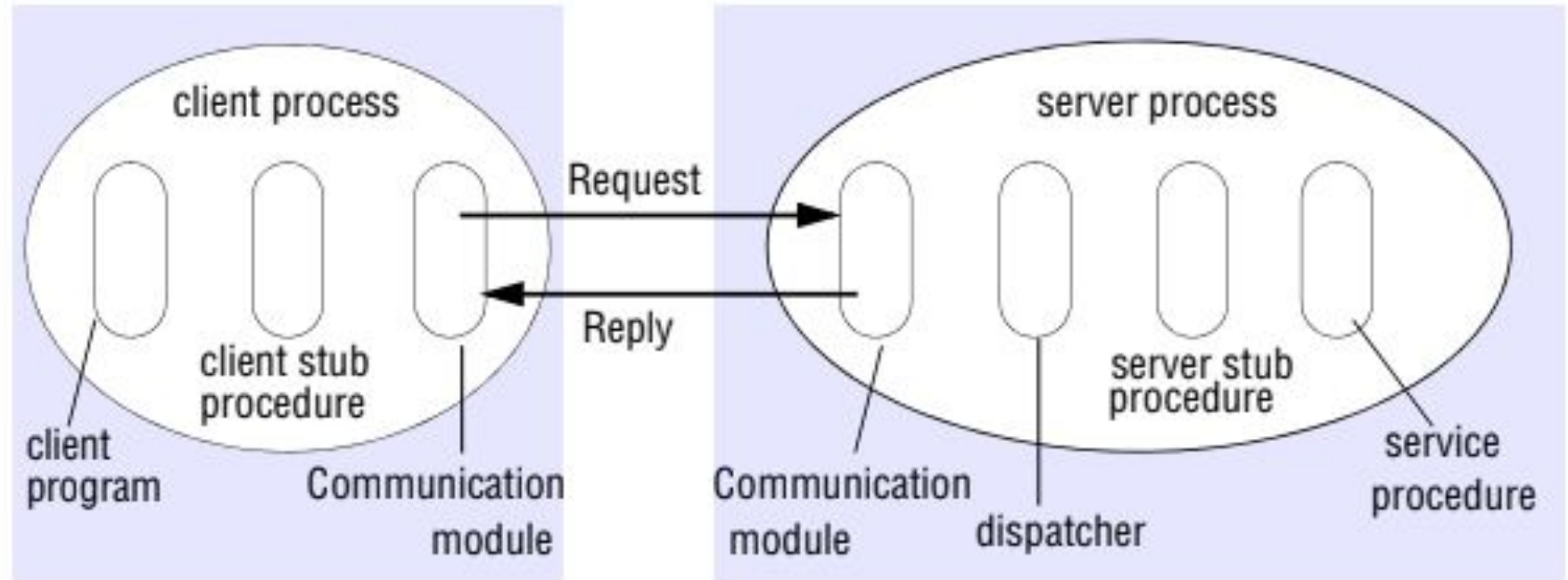
- the earliest and perhaps the best-known programming model for distributed programming.
- It allows client programs to call procedures in server programs running in separate processes and generally in different computers from the client.



RPC and RMI?

- A remote procedure call is very similar to a RMI in that a client program calls a procedure in another program running in a server process
- Server may be clients of other servers to allow chains of RPCs.
- A server process must define in its service interface the procedures that are available for calling remotely
- RPC, like RMI, may be implemented to have one of the choices of invocation semantics previously discussed (maybe, at-least-one, at-most-one).
- The contents of request and reply messages are the same as those for RMI, except that the *ObjectReference* field is omitted.
- The software that supports RPC does not support classes and objects.

Implementation of RPC



Client Side:

Client Stub Procedure: For each procedure in the service interface, the client includes a client stub procedure. This stub procedure emulates the behavior of a local procedure for the client application.

Marshalling: Instead of executing the call locally, the client stub procedure serializes or marshals the procedure identifier and arguments into a request message. Marshalling involves packaging data in a format suitable for transmission over a network.

Server Side:

Server Process: The server contains a dispatcher, server stub procedures, and service procedures.

Dispatcher: When a request message arrives at the server, the dispatcher selects the appropriate server stub procedure based on the procedure identifier in the request message.

Server Stub Procedure: The selected server stub procedure unmarshals or deserializes the arguments from the request actual implementation of the requested functionality. The server stub procedure invokes this service procedure, passing the unmarshalled arguments.

Service Procedure: It performs the necessary computations, operations, or tasks using the received arguments. Once the service procedure finishes execution, it produces a result (if any) that needs to be sent back to the client. The server-stub procedure serializes or marshals this result into a format suitable for transmission back to the client.

Communication Flow:

Client Request: Client stub marshals the procedure identifier and arguments into a request message and sends it over the network to the server.

Server Processing: The server's dispatcher identifies the appropriate server stub, which unmarshals the arguments and calls the corresponding service procedure.

Server Response: After executing the service procedure, the server stub marshals the return values into a reply message and sends it back to the client.

Client Response Handling: The client stub unmarshals the results from the reply message, allowing the client application to process the results as if it were a local **call**.

Events and Notifications

- One object can react to a change occurring in another object
- Notifications of events are essentially asynchronous

Distributed Event-Based Systems

- Distributed event-based-systems allow multiple objects at different locations to be notified of events (actions on objects) taking place at an object.
- Publish-subscribe paradigm:
 - an object that generates events publishes the type of events that it will make available for observation by other objects
 - objects that want to receive notifications from an object that has published its events subscribe to the types of events that are of interest to them.
- Different event types may, for example, refer to the different methods executed by the object of interest

Tasks

Write short notes on:

- Distributed File System.(page 298)
- SUN NFS(page 311)
- Name services and Domain Name System(Page 343)

Source: Distributed Systems Concepts and Design, 4th Edition by George coulouris, Jean Dollimore, Tim kinderg

Distributed File System

- File system performs organization, storage, retrieval, sharing and protection of files.
- Distributed File System provides storage and retrieval of files in distributed environment.
- Users and storage devices of DFS are physically separated.
- It implements a common file system that can be shared by multiple autonomous computers

Goals of DFS

DFS has two important goals:

- **Network Transparency**:: Users not aware of location of files.
- **High Availability**:: System failures or failure in regularly scheduled activities should not result into unavailability of files.

Distributed File System Requirements

Many of the requirements in the design of distributed services are as follows:

- Transparency
- Concurrent File Updates
- File replication
- Hardware and Operating System Heterogeneity
- Fault Tolerance
- Consistency
- Security
- Efficiency

Transparency

Following form of transparency should be partially or wholly addressed by the distributed file services:

- **Access Transparency:**

- Client programs should not be aware of distribution of files.
- A single set of operation should be provided for access to local and remote files.
- Programs written to access local files must be able to access remote files without modification

- **Location Transparency**

- Files or group of files can be relocated without changing their path names, and user programs must see same namespace whenever they are executed

Transparency

- **Mobility Transparency:** Client programs should not be changes when the file is moved from one location to another
- **Performance Transparency:** Client programs should continue to perform satisfactorily while the load varies on the service within specified range.
- **Scaling Transparency:** The file service should be expanded by incremental growth when the load also grows.

Concurrent File Updates

- Changes to a file by one client should not interfere with the operations of other clients simultaneously accessing or changing the same file.

File Replication

- In a file service that supports replication, a file may be represented by several copies of its contents at different locations.
- It has two benefits:
 - Quicker access
 - Fault tolerance

Hardware and Operating System Heterogeneity

The service interface should be designed in such a way that client and server software can be implemented for different operating systems and computers.

Fault Tolerance

- File service need to be functional even on client or server failures.

Consistency

One-copy update semantics

- all clients see contents of file identically as if only one copy of file existed
- if replication or caching is used: after an update operation, no program should observe a discrepancy between data in cache and stored data

Consistency Across Copies: Regardless of the presence of multiple replicas or cached versions of a file, any changes made to the file should be propagated uniformly and consistently across all copies. This ensures that all clients see the same content regardless of which copy they access.

Synchronization after Updates: After an update operation (such as a modification, write, or edit) is performed on a file, the system ensures that no program or client observes any discrepancies between the data stored in its local cache and the actual stored data. This means that changes made should be accurately reflected and synchronized across the entire system, including caches and replicas.

Discrepancy Prevention: The system guarantees that after updates, all cached or replicated data reflects the latest changes, avoiding any inconsistencies or differences between what is stored locally and the authoritative version of the data.

Security

There should be :

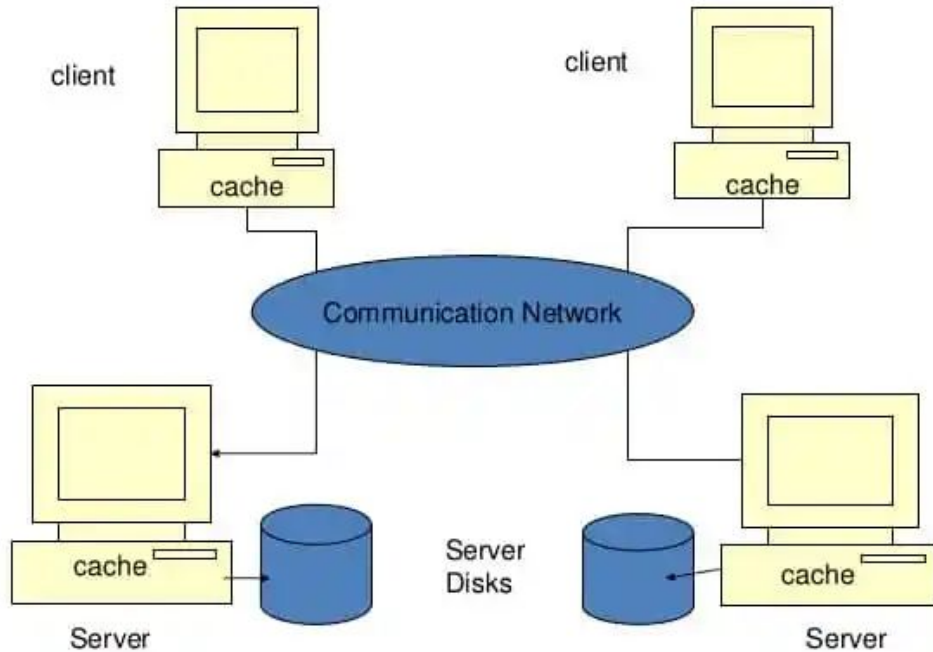
- client request authentication
- Encrypted data

Efficiency

Must be efficient in terms of:

- Latency of file accesses
- Scalability (e.g., with increase of number of concurrent users)

Architecture of DFS



Architecture of a distributed file system: client-server model

File Servers: Dedicated to storing and perform file access operations

Clients: They access files stored on the servers.

Client machines can be equipped with local disk storage that can be used for caching remote files or as a storage area.

The two important services in DFS are:

- Name Server
- Cache Manager

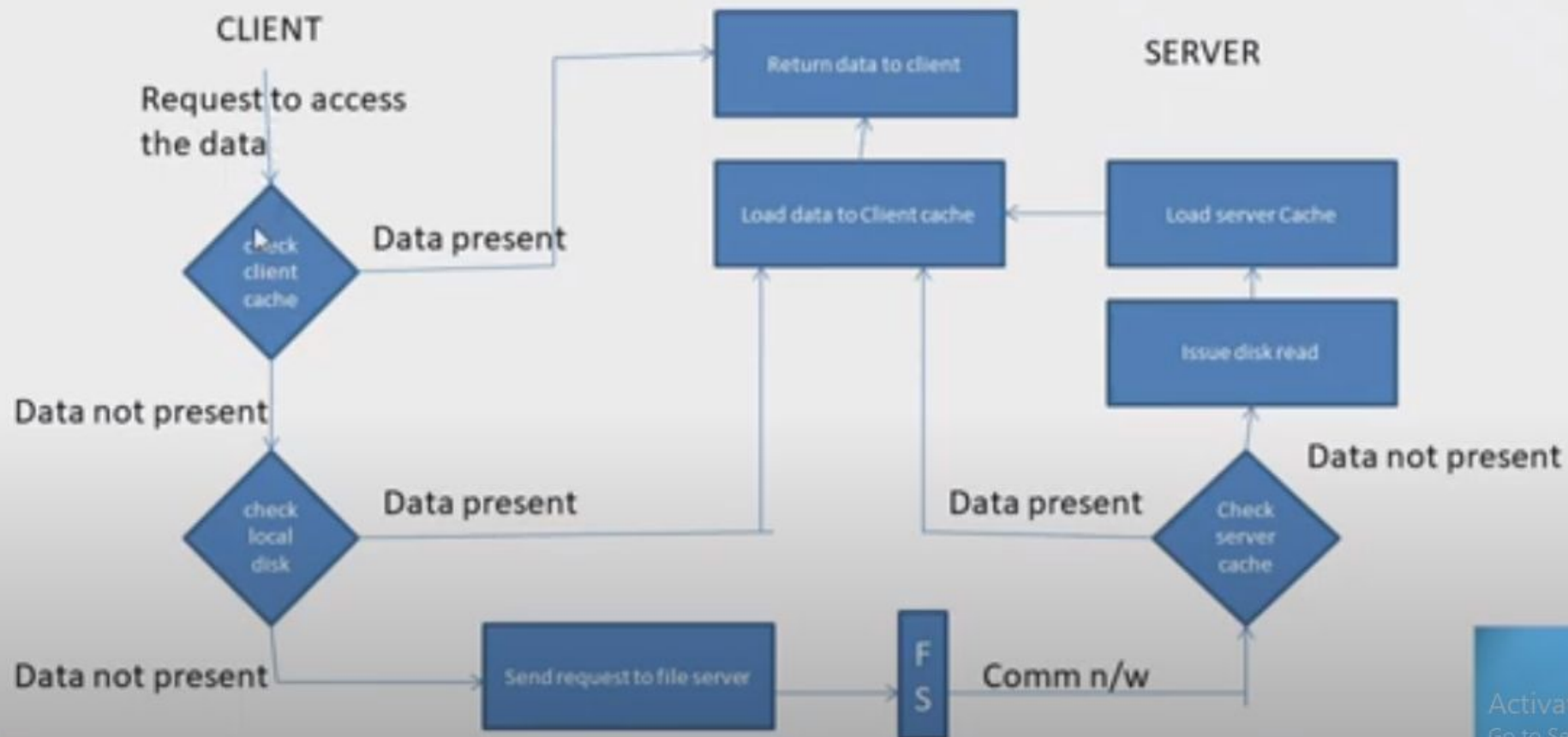
Name Server

- It is a process which maps names specified by users to stored objects as files or directories.
- This is also known as name resolution and it occurs when a process refers to a file or directory for the first time

Cache Manager

- It is a process that implements file caching.
- In file caching, a copy of file stored at a remote file server is brought to clients machine when referenced by client.
- Subsequent access to the same file by the client can be fetched from client's cache instead of getting it from remote file server thus reducing network latency.
- Cache managers are present both at clients and servers. At server side, cache managers cache files in the main memory to reduce disk latency.
- If multiple clients cache the same file and try to modify it, then the copies become inconsistent.
- To avoid this inconsistency, cache managers at client and server should coordinate with each other during data storage and retrieval operations.

Data access actions in DFS



Stateful vs Stateless file servers

- Stateless file servers do not store any session state.
- This means that every client request is treated independently, and not as a part of a new or existing session.
- Stateful servers, on the other hand, do store session state

Difference between Stateful & Stateless

Parameters	Stateful	Stateless
State	A Stateful server remember client data (state) from one request to the next.	A Stateless server does not remember state information.
Programming	Stateful server is harder to code.	Stateless server is straightforward to code.
Efficiency	More because clients do not have to provide full file information every time they perform an operation.	Less because information needs to be provided.
Crash recovery	Difficult due to loss of information.	Can easily recover from failure because there is no state that must be restored.
Information transfer	The client can send less data with each request.	The client must specify complete file names in each request.
Operations	Open, Read, Write, Seek, Close	Read, Write

Name Service

a name service plays a crucial role in distributed systems by enabling users or applications to refer to objects by a name or identifier rather than their specific addresses or locations within the network. Some important aspects of name services include:

Object Identification: In a distributed environment, users or applications often need to interact with various objects or resources. Name services provide a way to refer to these objects using human-readable names or identifiers, abstracting the underlying network details.

Facilitates Communication: Users or applications within a distributed system cannot communicate directly unless they can identify or name each other. Name services provide a mechanism for identifying and locating entities within the system.

Binding: The association between a name and an object or resource is called binding. The name service maintains these bindings, allowing users to access objects by their assigned names.

DNS (Domain Name System): One of the most widely known examples of a name service is DNS. It maps domain names (like `www.example.com`) to corresponding IP addresses (such as `192.168.1.1`) of host computers on the internet, enabling users to access websites by domain names rather than remembering IP addresses.

Name Resolution: Name services perform name resolution, translating user-friendly names or identifiers into the actual network addresses or attributes required to establish connections or access resources.

Example: In the given example, the name service (DNS) maps the domain name "`www.imm.dtu.dk`" to the specific IP address "`192.38.82.230`" associated with the IMM web server. This allows users to access the web server using the domain name instead of the numeric IP address.

Name Service

- A name service is a collection of naming context
- Naming context is the set of bindings between textual names and attributes for objects
- Major operation of a name service is a name resolution.

URL

`http://www.cdk5.net:8888/WebExamples/earth.html`

DNS lookup

Resource ID (IP number, port number, pathname)

`55.55.55.55`

`8888`

`WebExamples/earth.html`

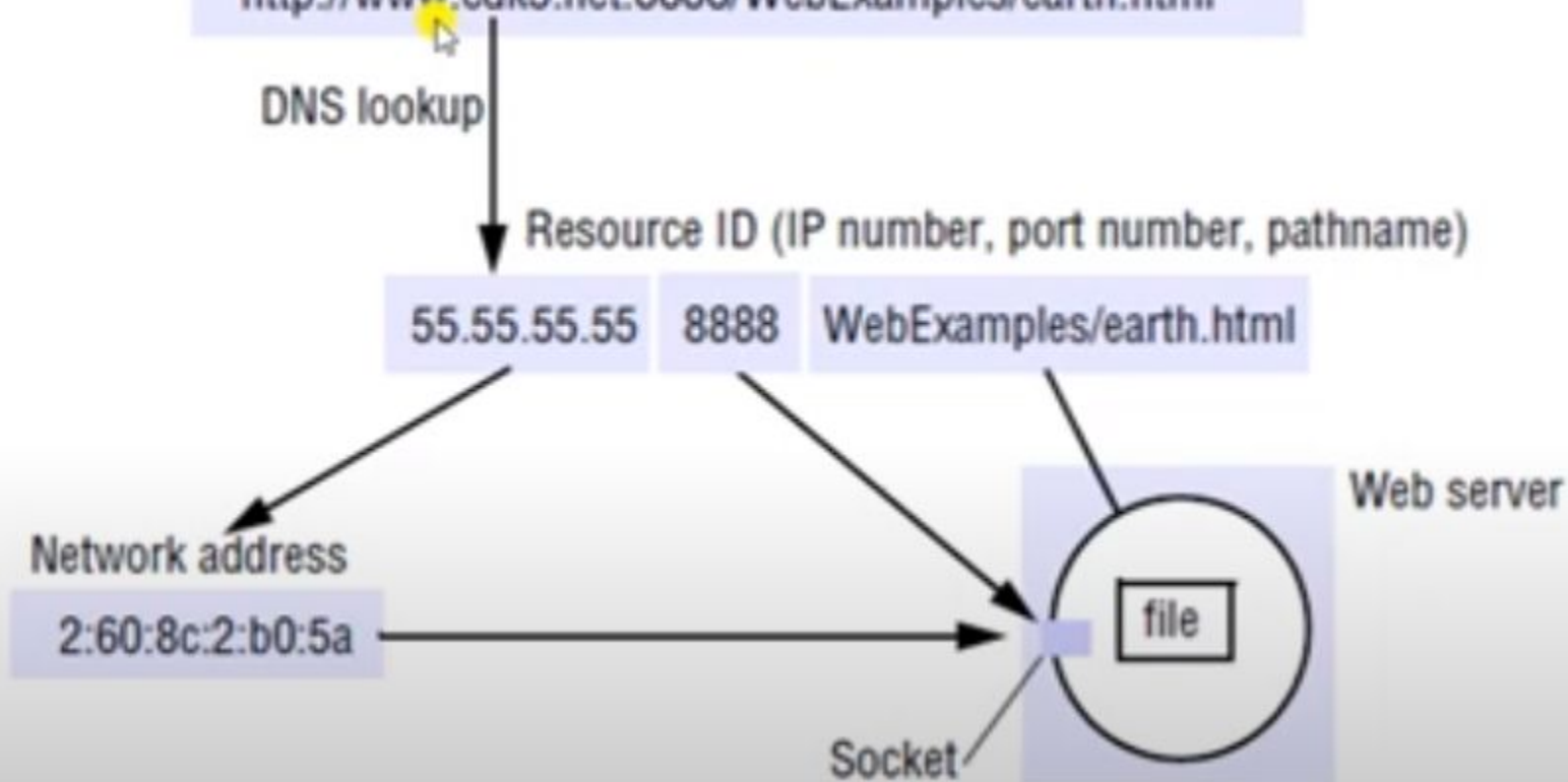
Network address

`2:60:8c:2:b0:5a`

Web server

file

Socket



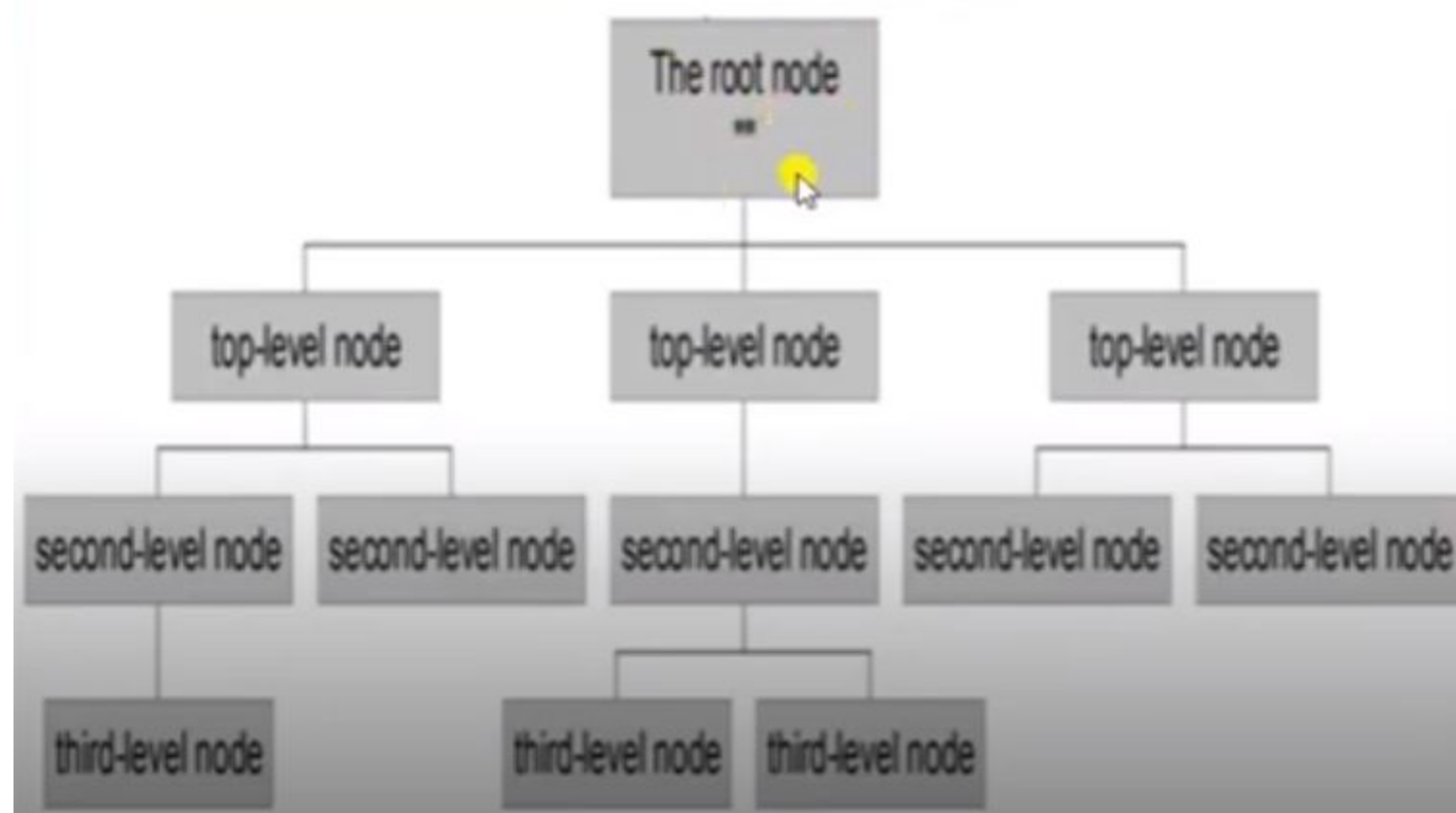
Name Service and Domain Name System(DNS)

- The domain name system is a name service design which performs name to IP mapping.
- The DNS is partitioned both organizationally and geographically.

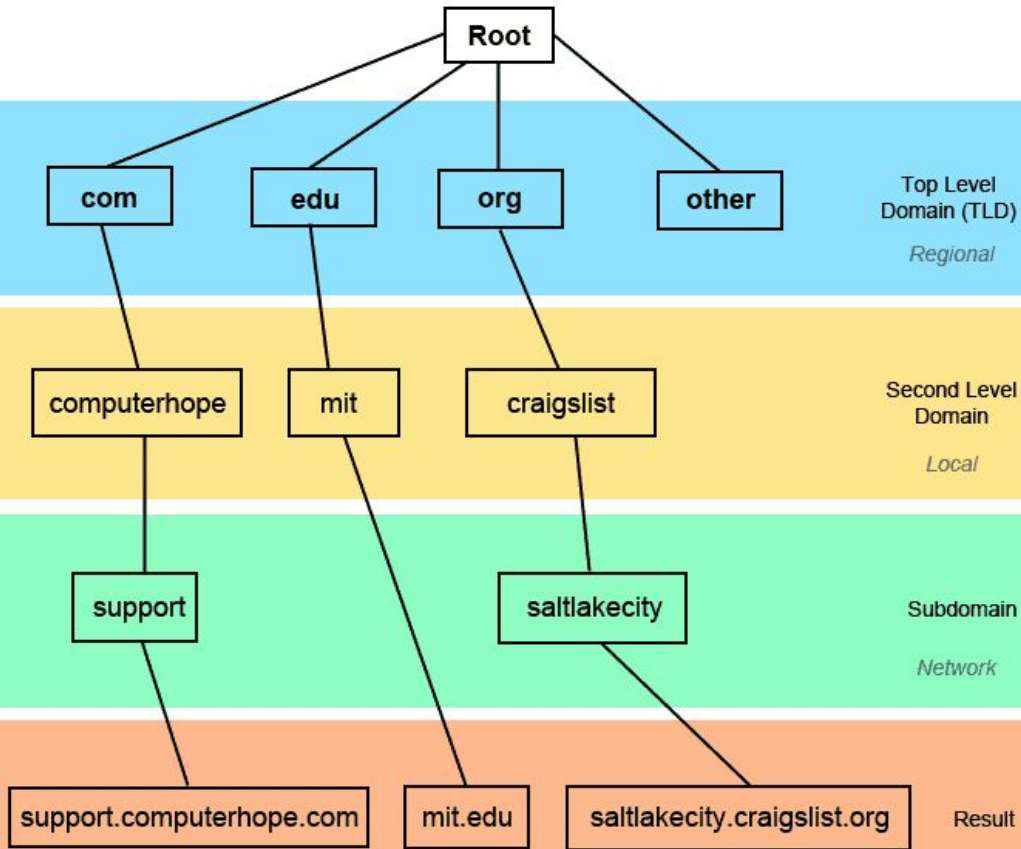
com	Commercial organizations
edu	Educational institutions
gov	Government institutions
int	International organizations
mil	U.S. military institutions
net	Networking organizations
org	Non-profit organizations

Name Service and Domain Name System(DNS)

- Every country has its own domain
 - us - United States
 - uk - United Kingdom
 - in - India
 - Np - Nepal
- DNS names are not case sensitive and also it has hierarchical structure



Domain Naming Hierarchy



Query Initiation: User enters a domain name (e.g., www.example.com) in a web browser, triggering a DNS query.

Local DNS Resolver: User's device contacts a local DNS resolver, which checks its cache for the domain's IP address.

Root DNS Servers: If not found in the cache, the resolver contacts root DNS servers (13 sets worldwide), providing information about TLD servers.

Top-Level Domain (TLD) Servers: Resolver reaches the relevant TLD server based on the domain extension (.com, .org, etc.), which holds information about second-level domains.

Authoritative DNS Servers: TLD server directs the resolver to the authoritative DNS server responsible for the specific domain.

IP Address Retrieval: Authoritative server provides the IP address associated with the requested domain.

Response and Caching: Resolver caches the IP address for future use to expedite subsequent queries for the same domain.

Time-to-Live (TTL): DNS records have a TTL indicating how long the information can be cached before expiration.

IP Address Delivery: User's device receives the IP address, enabling connection with the associated web server, allowing access to the website or service.

HOW DNS WORKS

